Sonder

A trip-planning system built by people tired of trip-planning systems

---

1. Abstract

The idea for Sonder came from a frustration our lead has carried for fifteen years of planning his own trips. Every existing platform optimises for the wrong thing. TripAdvisor surfaces what coach tours stop at. Google Travel knows the budget but not the mood. Instagram knows the mood but not whether a saved spot is closed on Tuesdays. Airbnb solves where you sleep and stops there. None of them solves the harder problem of *who you travel with*. Solo travellers get nothing. Couples get hotel suggestions. Anyone who has wanted to share a trip with a stranger they would actually click with falls back to hostel bars and dating apps repurposed badly.

Sonder is the platform we wanted to exist. It does three things together that no incumbent does together: it generates a day-by-day itinerary that names *real* places — the café with the rosé-coloured awnings on Largo do Carmo, not "explore the old town" — grounded in a hand-curated venue corpus; it matches solo and couple travellers against a measurable compatibility surface rather than free-text bios; and it sustains a believable social layer through the cold-start phase by seeding a population of LLM-designed synthetic travellers who post, open trips, and message real users about their plans.

The engine underneath is a multi-provider language-model stack. Anthropic Claude runs primary, OpenAI fallback, split across two tiers: Haiku for fast persona voice, Sonnet for deep generation, plus a validator tier on either side. Image generation handles synthetic-persona portraits. Voice synthesis powers in-chat audio. A 1536-dimensional embedding space holds destinations, activities, traveller personalities, and emotion-anchor vectors in a single coherent index. The pipeline streams day-by-day so a user sees day one of an itinerary while later days are still being written, and a validator engine catches assistant-voice leakage, hallucinated venues, and broken persona consistency before any of it reaches the screen.

This report walks through the data the system runs on, the models that drive it, the architecture that runs it, the validator stack that keeps it in character, and the integration battles the team fought to get there.

---

2. The data underneath everything

2.1 Four planes of data

Sonder is not a model trained on a static dataset. It is a live system that produces and consumes data across four planes simultaneously.

- **The reference corpus** — destinations and activities curated by hand, embedded once, queried thousands of times. Roughly fifty cities, around five hundred activities. Hand-curated rather than scraped because every scraped travel corpus reads like a search-engine results page.
- **The synthetic traveller population** — 192 solo travellers and 18 couples, each one designed by a language model under tight diversity constraints. Each carries a written backstory, a voice anchor, two or three quirks, an inferred emotional signature, and a stylised portrait. They live in the same vector index as the destinations.
- **The operational data plane** — what real users produce as they use the product: profiles, generated itineraries, chat sessions, journal entries, social posts, shared-itinerary negotiation history.
- **Telemetry** — every language-model call instrumented, every validator outcome logged, every match score recorded with its full feature breakdown. This is the substrate for phase-two ranker learning.

2.2 The frameworks we built on

The team did not invent the psychology of travel. Three pieces of prior work shaped the foundations.

Push-Pull Motivation Theory. Introduced into travel research by *Dann (1977)* and extended by *Crompton (1979)*, the theory frames travel as the product of two distinct motivational forces. Push factors are intrinsic and traveller-side — why a person leaves home. Pull factors are extrinsic and destination-side — what attributes of a place draw them specifically. We implemented this as a twelve-dimension persona space: six push, six pull, each defined by a substring-matched keyword set. The keyword-list approach matters because it makes the persona inference auditable — when the system tells a user they score high on *escape_reset*, the team can point at the exact phrases that produced the score.

Push (why they travel)	Pull (what they want from the place)
escape_reset — disconnect, recharge, leave routine	*nature_outdoors* — landscapes, weather, physical settings
adventure_novelty — push themselves, first-time experiences	*culture_history* — heritage, museums, local arts
connection — share with the people they love	*food_drink* — local cuisine, regional specificity
reflection — process, gain perspective	*nightlife_social* — bars, clubs, live music
curiosity — go deeper than the guidebook	*comfort_luxury* — refined service, high-end stays
prestige_reward — milestone trips, dream destinations	*exploration_local* — neighbourhoods, daily-life immersion

GoEmotions. *Demszky et al. (2020), arXiv:2005.00547* — a dataset of 58,000 Reddit comments labelled across 27 fine-grained emotion categories. Sonder does not ship the 58,000 labelled rows. The system uses the label vocabulary as anchor vectors. Each of the 27 emotions gets a one-line tone-anchored gloss — *realization* is *"a quiet click — coming to understand something"*, not a dictionary definition — and the team embeds those 27 glosses once at process start using the same 1536-dimensional embedding model that powers the rest of the system. A user's free-text input is then classified by cosine similarity against the anchors. The result is defensible emotion scores in the same vector space as every other signal in the product. The tone-anchored gloss choice is deliberate: dictionary-style definitions cluster too tightly in the

embedding space, while tone anchors crisp it.

An eight-key emotional-signature taxonomy. Synthesised by the team specifically for travel personas — eight archetypes (*story_collector, reset_seeker, aesthetic_pilgrim, depth_diver, energy_chaser, ritual_keeper, quiet_observer, threshold_walker*). The inferrer picks one key per user from the GoEmotions distribution plus the structured persona answers. The signature becomes private framing for every persona-voiced surface; the key itself is never shown — only the derived tone phrase like *"soft afternoon energy"* surfaces in the UI.

The personality stack is three lenses at three granularities: 27-label fine emotion → 12-dimension push-pull motivation → 8-key voice signature. Each consumer in the system reads the lens appropriate to its task. The chat-reply prompt reads the signature; the matcher reads the PPM space; the persona-reveal copy reads all three.

2.3 The travel APIs we integrated and the failure modes each one had

A travel product without real data about real places is fiction. The team integrated five external APIs, and every single integration revealed a failure mode that needed engineering around.

Wikipedia REST API + Wikimedia Commons power destination overview paragraphs and primary imagery. Wikipedia's article-summary endpoint returns the lede paragraph plus the article's infobox image, which is sourced from Wikimedia Commons under permissive licensing. The integration looked trivial until it was tested on a region rather than a city. For *"Patagonia"*, the infobox image is a Wikimedia Commons SVG showing southern South America with the region shaded gold. The cinematic reveal screen was rendering Wikimedia location maps as the hero photo on every region-shaped query. The fix took an evening: a URL-substring rejection filter that drops any `.svg` (almost always maps, flags, or coats of arms) plus any URL containing *map*, *karte*, *location*, *locator*, *satellite*, *topographic*, *flag_of*, *coat_of_arms*, or *seal_of*. The fix shipped alongside a 14-day client-side cache, with the cache key bumped so users with already-cached maps would refetch on next load.

Pixabay is the fallback when Wikipedia returns nothing usable, and the source for the cinematic destination-reveal montage. Pixabay's photography library is contributor-uploaded and licensed under the Pixabay Content License — commercial use without attribution. The integration required server-side proxying because exposing the key client-side would burn rate limits to abuse. It required a country-less retry path because *"Patagonia Argentina"* sometimes returns zero hits while *"Patagonia"* returns thirty. It required an in-process cache keyed by query and count so that a refresh of the reveal screen does not re-pay quota for the same five photos. The cache is what makes the montage — five photos cycling every 1.2 seconds with a slow zoom — feel free on subsequent views.

The two image sources are deliberately layered. Wikimedia gives us editorial-quality imagery when available; Pixabay backfills with destination-shaped travel photography for anything Wikimedia's curated infobox does not cover. Neither requires an attribution string baked into the UI under its license, which keeps the interface clean.

OpenWeather provides current conditions by latitude and longitude. Cheap, fast, well-behaved.

Nominatim (OpenStreetMap) handles geocoding from city/country tuples to coordinates. OSM's terms of service request no more than one request per second from a single source. The integration wraps Nominatim in a process-wide token-bucket rate limiter and a 30-day disk cache.

ExchangeRate API handles currency conversion at the input boundary. All internal cost fields are USD; conversion happens once when the user submits a budget. The integration has a 3-second timeout and a hardcoded fallback table for thirty currencies so the product does not break when the free tier flakes overnight.

A consistent pattern across all five: **cache aggressively, fail soft, never let a partner outage break the product.** Wikipedia down? Use Pixabay. Pixabay down? Fall back to a gradient hero. OpenWeather slow? Skip the weather line. Nominatim rate-limited? Cache hit. For a product where users are mid-flow planning the trip of their year, the user never sees an error screen because a third party had a bad afternoon.

2.4 The synthetic-traveller corpus — why we built a population from scratch

The matching surface needed a candidate pool, and real users would not exist on day one. The honest options were either to launch with an empty matching screen or to seed a population. We chose to seed, but the construction matters.

The diversity matrix is locked: sixteen cities across five continents, three age buckets (20-30, 30-40, 40-50), two genders in a 50/50 hard-locked split, two personas per cell. Total: 192 solo travellers. A separate matrix produces 18 male-female couples for couple-mode matching. The two-per-cell density was chosen after a real production incident detailed in section 3.

Each persona is generated through a **two-stage blind-writer pipeline**. The first language model receives only what a novelist would receive: the city, the age bucket, the gender, and four persona-question option keys to pick from. It writes the character — name, voice anchor, small-thing answer, quirks, appearance descriptor. It never sees the PPM dimensions, the emotional-signature taxonomy, or any matching feature names. The second stage, the inferrer, then runs the written persona through exactly the same pipeline a real user would, assigning PPM dimension labels and an emotional signature blind to what the writer was *supposed* to produce.

This is the most consequential design decision in the system. A language model with the answer key in front of it writes to that key. A language model that has to guess writes something honest. Portraits are generated from the appearance descriptor only — painterly, explicitly not photorealistic, which biases the population the right way for the *"Sonder Curated"* disclosure pattern. End-to-end cost is roughly $2-4 in API calls per full seed run.

2.5 Schema realities — what is actually in the data

Free-text length distributions across the synthetic corpus:

Field	Median (words)	Range
Voice anchor	20	8-40
"Small thing" free text	14	6-35
Quirks (concatenated)	16	8-30
Embedding text (the full string vectorised)	110	40-220

Two missing-value paths bit the team in production and are worth surfacing as concrete examples of the system's fail-soft posture.

Per-question answer salience. A per-user weighting that boosts the matching contribution of free-text answers the user revealed more on. When absent — older profiles created before the field was added — the matcher falls back to uniform weighting. The fail-open path in the ranker ensures older profiles still get matches.

Gender on synthetic personas. A near-disaster. After the same-gender hard filter for solo travellers shipped, the matching surface immediately started returning zero candidates. The cause: gender was on the synthetic-persona schema, but the Pinecone metadata write step in the seed script — written months earlier — was only storing gender in the log-preview output, not the actual metadata dictionary. Every record in production had no gender field. The filter saw zero qualifying candidates, and the safety default of *"fail open to mixed matching"* meant the filter never fired at all. Diagnosis came at 2am. The fix was a metadata-only patch script that rebuilt the deterministic diversity matrix and called `index.update(set_metadata={"gender": ...})` per profile id, without re-paying language-model or image-generation cost. The script ran against production and patched 96 records in under a minute. The seed-script code path was then fixed so future re-seeds would not repeat the bug.

2.6 The eight-key emotional signature explained as a layer

The eight-key emotional-signature taxonomy is the third lens in the personality stack — coarser than GoEmotions, more identity-shaped than PPM. The inferrer picks one key per user based on two pieces of evidence — the GoEmotions distribution over the user's free text and their structured persona answers — and attaches a confidence level (low / medium / high). The signature is then used as private framing for every persona-voiced surface. The system is explicitly instructed never to use the taxonomy keys in output text, because they are internal labels rather than language a real person would naturally use.

2.7 The drift patterns we caught in generated content

Two recurring failure modes surfaced in the first weeks of running the synthetic-agents loop.

Sales-register drift. Early synthetic open-trip notes read like travel marketing copy. *"Join me for an unforgettable adventure!" "Looking for like-minded wanderlust souls."* The fix was to put forbidden-openers lists with explicit bad examples directly into every persona-voiced system prompt. Showing the model exactly what bad output looks like — not just describing it — turned out to be substantially more effective.

Question-loop drift in chat. Personas would drift into interrogating users after about turn seven of a session. Real people do not text like this. The fix was a runtime instruction inserted into the prompt when two or more of the persona's recent turns ended with a question mark — a *"breathe hint"* telling the model to react, observe, or share something small instead.

Typography drift caught the worst. A user pointed out a message reading *"the street pad thai is genuinely different from restaurant versions , way more char on the noodles. there's a few stalls near the floating markets..."* — stray space before the comma, singular contraction with a plural subject. The first fix is universal: a regular expression collapses whitespace before punctuation in every persona-voiced message before broadcast. The second went into the system prompt as a *"casual ≠ sloppy"* rule: *"there are a few stalls", not "there's a few stalls". A real person texting still hits agreement and spacing.*

2.8 What the validator engine watches for

Every persona-voiced surface runs through a validator stack that watches for five regression categories:

Category	What it catches
Assistant-voice leakage	*"How can I help you?", "I'd be happy to..."* — chatbot register bleeding through
AI / tooling leakage	*"As an AI", "I'm a language model"*
Memory contradiction	Persona claims to have been somewhere, then later denies it within the same chat
Token-level failure	Empty generations, repetition stutters, malformed JSON
Internal-taxonomy leakage	Persona uses the system's own labels (*push*, *pull*, *motivation*) instead of behavioural language

A deterministic local pre-check runs first — regex-based, instant, no API cost. The LLM critic only runs if the local check is inconclusive. The two-layer arrangement kills the cheap failures pre-LLM and reserves model spend for the genuinely ambiguous cases.

2.9 How we built the vector database

The Pinecone vector index is the single most queried piece of infrastructure in the system. Every itinerary generation, every co-traveller match, every destination lookup hits it.

Single index, three namespaces. A managed Pinecone serverless index hosts three logical partitions: `destinations` for city contexts, `activities` for venue contexts, `cotravellers` for traveller personality vectors. Namespaces share the index and dimension but isolate query scope, so a co-traveller search never accidentally returns a destination. Configuration stays centralised.

1536-dimensional embedding space, unified across every surface. Every vector — destinations, activities, traveller personas, GoEmotions anchor glosses, the user's query text at retrieval time — is produced by the same OpenAI `text-embedding-3-small` model. Keeping all content in a single embedding

space means cosine queries are coherent across namespaces. The dimension constant lives in one environment variable so a model change is one variable update plus an index recreate.

Metadata-driven filters at query time. Every vector carries a metadata dictionary alongside the embedding. Cotraveller records carry travel style, gender, age, location, archetype, persona answers (JSON-encoded), and the raw embedding text. Activity records carry category tags, cost in USD, duration in hours, destination identifier. The matching pipeline's hard filters — budget feasibility, style match, gender for solo travellers, avoid-list veto — all run against metadata, *before* re-ranking rather than after. The qualified pool reaches the feature ranker; the top three surface.

Embedding text shape matters. The string vectorised for a traveller is a concatenation: voice anchor, small-thing free text, quirks, persona-answer labels rendered as human-readable text. For destinations and activities, the embedding text is the full curated context block — 200 to 400 words describing what the place feels like rather than dry facts. Embedding the feel of a place lets cosine similarity surface emotional matches that a fact-based embedding would miss.

Metadata-only updates are the template for schema evolution. Pinecone supports `index.update(set_metadata={...})` without re-embedding. The team built a backfill script that rebuilt the deterministic diversity matrix, computed each existing profile's ID, and patched metadata in place. Pattern generalises to any future schema-only field addition.

---

3. The systems we built and the design choices behind them

The team identified three problems to walk through in detail, each architecturally different from the others: a retrieval-grounded generation problem, a validator-gated conversational generation problem, and a feature-pipeline ranking problem. For each, we describe what the system actually does, the alternative we considered, and the trade-off that made us choose what we built.

We did not run controlled A/B benchmarks against the alternatives — none of them were implemented as production systems. The comparisons are architectural rather than measured. Where claims sound quantitative, they reflect engineering judgment about the failure modes each design would have, not benchmark numbers.

3.1 Itinerary generation — RAG pipeline grounded in a curated corpus

What the user is doing. Submitting a destination, dates, a budget, free-text persona answers, plus must-haves and things to avoid. Receiving back a day-by-day itinerary that names *specific* places, respects every negative constraint, fits the user's pace and budget, and includes a short *"why this for you?"* rationale in the persona's voice on every activity.

What we built. A retrieval-augmented generation pipeline grounded in the hand-curated venue corpus. The user's persona is inferred across three pipelines running in parallel — a text embedder, the GoEmotions cosine classifier over their free text, and a language model that picks PPM dimension labels from a closed

tool-use enum. None of the three sees the other two's outputs while running. The destination is selected from the city vector store. Activities for that destination are retrieved from the activity vector store, ranked through a feature pipeline (cosine score, persona-question salience-weighted overlap, ordinal pace fit, budget feasibility, interest overlap), then handed to the itinerary-generating language model.

The model receives the user's persona, the retrieved real activities, and the emotional-signature framing. The model never sees the ranker weights, the cosine scores, any other user's persona, or the validator's prompt. Output streams day-by-day. The streaming parser yields a parsed day the moment its closing brace lands, so the user can read while the model writes.

A separate validator engine critiques the result across seven categories — budget fit, pacing realism, must-haves covered, avoid-list respected, day-sequence logic, activity specificity, feasibility risk. Outputs that fail can go through a refinement loop of up to three regeneration attempts, with the persona re-embedded each time so the model receives a fresh query rather than grinding on the original.

The alternative we considered. Pure prompt LLM with no retrieval, no validator, no refinement, no streaming. The user persona and constraints rendered into a single prompt; the language model writes the itinerary cold.

How the two compare across the product dimensions that matter.

Dimension	Champion (RAG + ranker + validator)	Alternative (prompt-only LLM)
Venue grounding	Real activities from the curated corpus	Invented from training distribution; subtly wrong on less-represented destinations (closed venues, mis-described neighbourhoods)
Negative-constraint enforcement	Hard pre-filter at retrieval; deterministic	Soft constraint inside the prompt; LLM may or may not honour
Budget feasibility	Hard pre-rank filter on raw per-day budget	No enforcement; honour rate degrades on luxury-tier prompts
Time to user-visible first day	Streaming day-by-day	Full buffer wait before anything renders
"Why this?" rationale	Grounded in the actual ranked activity record	Reasoning purely from internal patterns; ungrounded
Refinement on quality regression	Validator-gated refinement loop with up-to-three attempts	No correction path

The qualitative judgements above are architectural, not measured against a controlled A/B run. They reflect engineering reasoning about each design's failure modes.

The integration battle behind this. The user-initiated revise flow originally reused the three-attempt refinement loop from the initial generation pipeline. Each iteration is a full regen plus validate. Wall time on a

revise was hitting two to three minutes against Cloudflare's edge function proxy timeout, leaving the page hanging with no error and no result. The rebuild moved revise to a single-pass classifier-routed pipeline with SSE streaming so days appear as they parse. Small-scope edits got routed to a day-targeted prompt that regenerates only the affected days rather than the whole trip.

Trade-off. The champion approach requires ongoing curation of the destination and activity corpora — fifty cities and five hundred activities is a finite budget. When a user picks a city not in the corpus, the system falls back to *"invent plausible activities for {city}"* mode, which re-introduces the hallucination risk the RAG path was designed to eliminate. Automated corpus expansion is future work.

3.2 Chat replies — small-tier with validator-repair and edit-in-place

What the user is doing. Texting a synthetic persona inside Sonder. Expecting a reply in character — texting register, no assistant voice, no AI leakage, no semantic genericity, no contradiction of what the persona said three turns ago — within a perceived-real latency budget.

What we built. Chat replies route to the small language-model tier (Claude Haiku as primary, GPT-4o-mini under fallback). The persona's prompt layers three private framing blocks before the style rules: a hard trip-scope block (*"the trip is to Lisbon; never mention your home city, never suggest alternative destinations"*), a private psychology block where PPM dimensions and the emotional signature are passed as framing with an explicit instruction never to surface the words *push*, *pull*, *motivation*, *alignment*, or *friction* in output, and a per-turn breathe hint when recent turns have been too question-heavy.

The three pre-LLM fetches needed to assemble context — vector-store candidate lookup, itinerary fetch, message history — run in parallel rather than serially.

The architectural decision that makes the whole thing work: **the reply broadcasts to the user immediately, before validation.** A separate validator task runs asynchronously after broadcast. It first executes a deterministic local pre-check (minimum reply length, repetition detection, semantic-genericity score against a fourteen-stem filler set). If issues fire, a repair prompt rewrites the reply in a single pass under a fifty-word ceiling. If the repair changes the text, the persisted message is updated and the chat surface receives a *message-edited* event that swaps the bubble text in place. The user sees the reply land instantly and watches it quietly self-correct a moment later when needed.

How the two compare.

Dimension	Champion (small-tier + validator + edit-in-place)	Alternative (large-tier prompt, no validator)
Time to user-visible reply	Fast (small-tier inference)	Slower (large-tier inference is intrinsically heavier)
Cost per reply	Substantially lower (small tier)	Substantially higher (large tier)
Register failures (assistant voice, banned filler)	Caught by validator; repaired in place if needed	Ship to the user unchanged

Dimension	Champion (small-tier + validator + edit-in-place)	Alternative (large-tier prompt, no validator)
Memory contradictions	Regex-checked against established history	Uncaught
Operational simplicity	Higher complexity (validator stack per surface)	Simpler — one prompt per surface

The architectural bet is that running a small model with validator-driven repair beats running a large model without it on every product dimension *except* operational simplicity, and that the simplicity cost is worth paying once per surface to gain latency, cost, and quality on every reply.

The trade-off. The validator stack requires per-surface maintenance. Every conversational surface — chat, persona reveal, proposal evaluator, social posts — has its own critic prompt and category set. The single-large-model approach is operationally simpler but slower under the typing indicator, more expensive per reply, and any register failure ships unchanged.

3.3 Co-traveller matching — feature pipeline with weight-penalty learning

What the user is doing. Receiving a top-three list of potential co-travellers ranked by a number that honestly means *probability we'll click*, rather than a uniformly high cosine similarity that compresses every candidate into a narrow indistinguishable band.

What we built. A stage-based ranking engine that runs the same code path across three matching surfaces — co-traveller, destination, activity — with each surface declaring its own policy. Ten reusable scoring functions are available: raw cosine retrieval score, persona-question overlap weighted by per-question answer salience, emotional-signature exact match, pace ordinal distance, budget ordinal distance, travel-style exact match, two flavours of interest overlap, activity cost fit, pace-duration fit.

Hard pre-ranking filters fire before scoring. Budget feasibility uses a raw per-day budget cutoff with no fudge multipliers. Avoid-list veto removes anything matching a negative-constraint string. A travel-style hard filter ensures couples never see solo personas. A same-gender hard filter for solo travellers keeps solo women matched with women and solo men with men — a safety default for cold-strangers matching.

Per-user learning as weight penalties on the cost function. The team modelled feedback as a cost function with feature weights, where user feedback applies weight penalties that force the system to re-cost its own previous recommendations.

- **Explicit revise feedback** maps free text to ranker features. *"Cheaper"* maps to budget-fit features. *"Less packed"* maps to pace features. *"More local"* maps to interest-overlap features. The boost on matched features is positive; the implicit reduction on competing features is negative. The weight update applies with **decay across turns** — turn one full strength, turn two half, turn three a quarter, with a floor at one-eighth. The decay prevents oscillation when a user keeps pushing back across multiple revisions.

- **The self-correction loop.** A weight penalty is not a record-keeping update. After the new weights are written, the same generation pipeline re-runs from retrieval through ranking through generation through validation with the new weights applied. The system literally re-prices the candidates and regenerates the affected days. Revision history records every turn — feedback, scope, target days, dropped titles, added titles, which feature weights moved, by how much, and the validator's verdict.
- **Implicit chat signals** are extracted by a scanner that runs after every user message in a chat session. The scanner imports the same PPM keyword vocabularies the matching feature pipeline uses. Negation zones extend five words or to the next clause break, whichever comes first — *"I don't love crowded places"* does not boost crowded-tag interest. Sarcasm markers block boosts on `/s`, *"said no one ever"*, eye-roll emoji, or *"love how..."* clause openers.

The match score updates continuously as the conversation accumulates signal. The persona's reciprocal-approval probability reads the live score at the moment the user approves, so what a user reveals during a chat directly influences whether they end up matched.

How the two compare.

Dimension	Champion (feature pipeline + filters + learning)	Alternative (pure cosine retrieval)
Score interpretation	Spread across a meaningful range; usable directly as reciprocal-approval probability	Compressed into a narrow high-similarity band; denial only meaningful on extreme outliers
Hard safety filters (gender, style)	Enforced before re-rank	Absent — cosine ranks across all axes uniformly
Cross-style bleed (couples seeing solos)	Eliminated by hard filter	Common, because non-style axes carry high embedding similarity
Adapts to in-chat signals	Re-ranks per message via the signal scanner	Frozen at retrieval time
Adapts to revise feedback	Text-feedback path with decay applies weight penalties	None
Per-match explainability	Reasons + compatibility breakdown surfaced	Just a number

The architectural bet is that calibration matters more than raw retrieval precision for a matching surface, because the downstream behaviour (the persona's reciprocal-approval roll, the user's deny pattern) depends on score-as-probability honesty.

The pool-doubling firedrill. When the same-gender hard filter first shipped, top-three match scores within each gender dropped substantially because the filter had halved the effective candidate pool. With the persona's reciprocal-approval formula `p_approve = match_score`, lower scores produce more denies, and users reported seeing more denies than before. They were right. We had two options: fudge the probability formula (dishonest, breaks the calibration) or double the pool. We doubled the pool by bumping `PERSONAS_PER_SLOT` from one to two and running the seed with `--resume` so existing personas were not

regenerated. Generating the additional 96 personas cost roughly $4 in API calls. Scores within each gender lifted; the deny pattern returned to its prior shape; the calibration stayed honest.

Trade-off. The champion ships with equal-weight priors across features — one over N for N features — because the team has not yet earned data-grounded confidence in per-feature importance. This is the honest position but it leaves measurable signal on the table. The infrastructure to learn proper weights is in place: every shown candidate, every accept, every reject is logged with the full feature breakdown. Phase two will compute replacement gradients (the difference between accepted and rejected feature vectors) and learn per-user weights.

3.4 Group-style filtering — four user types as four products

The matching surface treats solo, couple, family, and friends as four different products. Trying to make one ranker serve all four would have meant compromising every one of them.

Style	Group size	Matching pool	Itinerary framing
solo	locked at 1	active (solo ↔ solo, same-gender filter applies)	Second-person singular; isolated instincts; counter-seating venues; 1-2 meet-others activities per day without forcing them
couple	locked at 2	active (couple ↔ couple, male+female by seed design)	*"You both"* framing; every overnight private; at least one slow shared activity per day; tables for two not bar-only
family	2-8 user-picked	**disabled**	Assumes kids present: kids-friendly menus, walking blocks under 30 minutes, dinner by 7pm, at least one kid-facing activity per day, apartment with kitchen access
friends	2-8 user-picked	**disabled**	*"Your group"* framing; one-table reservations for N; split-and-rejoin activities; nightlife on the table; apartment over N hotel rooms

Why family and friends matching is disabled. A family of four already has its travelling party. A group of friends already has its group. Surfacing strangers as *"companions"* to either makes no product sense. Both cases route straight to the shared-itinerary surface where the existing group plans the trip together.

Why the hard style filter exists. The ranker has a `style_match` feature that nudges the score, but a feature can only nudge. Without the hard filter at the route layer, couples were seeing solo personas as matches because of high embedding similarity on other axes. The filter drops cross-style candidates after retrieval and before re-ranking.

The couple-mode seed pool. Eighteen male-female couple personas exist because the singles pool — where every persona defaults to `travel_style = solo` — would have returned zero matches under the couple-mode hard filter. The couples seed script mirrors the singles blind-writer architecture but uses

couple-specific layering: voice anchor uses *"we"*, quirks describe the couple's dynamic (*"she plans, he wings it"*), portraits frame two people in candid posture rather than engagement-shoot framing.

3.5 The pattern across the three champions

One architectural decision shows up across all three champion approaches and the team would defend it more loudly than any other choice in the system: **information starvation as quality control.** The persona-inferring language model never sees the embedding vector or the GoEmotions output it will be merged with downstream. The validator never sees the user prompt the reply was responding to. The matcher's policy file never sees individual user data — only the feature names to score. Each component receives exactly the slice of context relevant to its narrow task; the merge happens downstream rather than co-prompted upstream. A language model with the answer key in front of it writes to that key. A language model that has to guess writes something honest.

---

4. How it actually runs

4.1 Deployment architecture — separated layers at scale

Sonder runs on four cleanly separated service planes. Each was chosen to be best-in-class for its responsibility rather than bundled into a single platform. The separation matters because it means scaling, monitoring, or replacing any one layer does not ripple through the others.

Frontend on Cloudflare Pages. The React single-page application is built statically and served from Cloudflare's global edge network. Static asset delivery happens at the edge with no cold start. A catch-all Cloudflare Pages Function proxies every `/api/*` request to the backend, keeping the SPA on a single domain. The single-domain constraint matters because the web-push service worker can only register against its own origin, and cross-origin token traffic risks leaking authentication state. The initial plan tried to handle this through simple HTTP-redirect rewrite rules. Cloudflare rejected them, because cross-origin proxying via redirect violates their terms. The edge-function rewrite handled it cleanly.

Backend on Render. A FastAPI process runs on a long-running container. The single process exposes REST routes, server-sent-event streams for itinerary generation and revision, WebSocket endpoints for chat and notifications, and runs the synthetic-agents background loop in the same lifespan. Render handles auto-deploy from GitHub main, TLS termination, and the health-check loop. The architecture is designed to scale to multiple replicas with one configuration change — the in-memory WebSocket connection manager becomes a Redis pub/sub channel via the `REDIS_URL` environment variable, so messages sent to one container reach WebSocket sessions on another. The abstraction was built in from day one.

Multi-provider language-model layer. The routing engine reads a per-tier provider preference from configuration. Small tier picks one primary; large tier picks another. Either tier's automatic fallback is the other provider. Each provider client carries its own model identifier so cross-provider failover cannot accidentally route an Anthropic model id to OpenAI or vice versa — a discipline that bit the team once during a Sonnet deprecation week.

Data stores split by access pattern. A managed Pinecone serverless vector index holds the three namespaces. A named-database Firestore instance holds operational state — user profiles, generated itineraries, chat sessions with messages as a subcollection, shared-itinerary negotiation logs, social posts and comments, journal entries. Firebase Storage holds binary assets — persona avatars and cached voice MP3s. Each store was chosen for its access pattern: Pinecone for cosine retrieval, Firestore for real-time listeners and small-document CRUD with strong consistency, Storage for blob serving with public-read URLs the frontend can render directly.

Observability split into two tools. Sentry handles error telemetry. PostHog handles product analytics. The two are deliberately separate because they answer different questions and live in different team workflows. Sentry catches stack traces, response-latency outliers, third-party-API failures, and the genuinely-broken events that pull an engineer into the codebase the same day. PostHog catches funnel conversion, retention cohorts, feature-flag rollouts, validator-stack telemetry, and the slower aggregate metrics that product decisions are made on. Sentry's interface is built for triaging incidents — group similar errors, mute noise, assign owners. PostHog's is built for cohort analysis and behavioural reasoning. Conflating the two would make both worse.

The Sentry feed is filtered: `Failed to fetch` TypeErrors are dropped (client lost internet — not a bug), and 4xx HTTP responses are dropped (404 on first profile fetch is the documented signal that we should create the profile; 401 during auth-state transitions is expected). Only 5xx responses and genuine uncaught exceptions reach the dashboard.

The point of the separation is **operational independence.** When Render redeploys, the frontend keeps serving from Cloudflare. When Cloudflare has a regional incident, the backend stays reachable for direct API calls. When Pinecone is slow, Firestore remains fast for the chat and trip-vault surfaces. When Anthropic rate-limits us, OpenAI picks up the load. Every layer can be replaced independently — Render swapped for another container platform, Cloudflare swapped for Vercel, Pinecone swapped for Weaviate — without rewriting the others.

4.2 The validator engine — guardrails that keep personas in character

The validator engine is the layer the team is most proud of architecturally. Every persona-voiced surface — chat reply, opener, proposal evaluator, persona reveal, social post, *"why this?"* explanation — runs through a configurable critic stack that catches assistant-voice leakage, AI tooling leakage, semantic drift, memory contradictions, and a long tail of register failures before the user sees them.

Five surface-specific validator prompts, each strict and category-scoped. The itinerary critic watches for budget fit, pacing realism, must-haves coverage, avoid-list violations, day-sequence logic, activity specificity (the *"swapability test"* — would this description apply to any city?), and feasibility risk. The persona-reveal validator watches for concrete observation versus generic framing, evidence fidelity to the user's actual inputs, no itinerary content bleeding into persona copy, specificity, and internal-label leakage. The cotraveller match-card validator watches for ranking grounding, evidence fidelity, specificity, tension awareness (a reveal that mentions both alignment and friction reads more honest than one that only flatters), internal-label leakage, and tone. **The chat-reply validator is the most active**, covering assistant

voice, AI leakage, semantic drift, token stutter, empty token generation, romantic-vibes detection, taxonomy leakage, unsafe content, bad conversation dynamics, and contradiction against established chat memory. The chat-reply repair prompt rewrites in a single pass under a fifty-word ceiling, preserving context but stripping anything the critic flagged.

The local deterministic pre-check runs first. Before any LLM critic call, a regex pass checks for minimum reply length (under three characters fires `empty_token_generation` and returns immediately, never hitting the LLM), repetition stutters, and a semantic-genericity score computed by counting matches against a fourteen-stem filler set. The genericity score is `base + matches × multiplier`, fires above a `0.80` threshold, and is also emitted as telemetry so the genericity distribution can be watched for drift. This local pass either kills bad replies outright or feeds its issue list to the LLM repair prompt as concrete *"validation issues"* context, so the rewrite is grounded rather than blind.

The trip-scope guardrails in persona prompts. Every persona-voiced system prompt layers three private framing blocks before the style rules. The first is the **hard trip scope** block: *"STAY INSIDE THIS TRIP. The trip is to {destination}."* plus the itinerary digest, with explicit prohibitions on mentioning the persona's home city, on referencing past trips, on suggesting alternative destinations. Without this block, the model drifts — a Paris-based persona on a Japan trip would open with *"I see you're interested in Lisbon too?"*, the model gravitating back to wherever its training data has more mass. The second block is **PPM as private framing**: push, pull, and motivation labels passed in as context with the explicit rule *"never say push, pull, motivation, alignment, or friction in output — turn those into concrete behaviours, opinions, scenes, instincts."* The third block is the **emotional signature**, framed as *"private emotional framing (never expose these words). Let this shape cadence, warmth, pacing, what you notice — never the vocabulary."*

The result is personas that talk about a specific trip without dragging in their own backstory or the system's internal taxonomy. The critic catches the cases where the model breaks scope anyway.

Memory-contradiction detection. The validator parses chat history with two regex templates — past destinations matching *"i've been to X"* / *"visited X"*, and past negations matching *"never been to X"* / *"haven't visited X"* — and flags any reply that contradicts the established timeline. This catches the specific failure mode where the persona *remembers* trips that never happened or denies trips it just claimed three turns earlier.

Async edit-in-place for chat replies. For chat specifically, the validator is fully off the critical path. The unvalidated reply broadcasts immediately, then `_validate_async` runs in `asyncio.create_task`. If repair changes the text, the persisted message is updated and a `message_edited` WebSocket event tells connected clients to swap the bubble text in place. Users see the reply land instantly and watch it quietly self-correct when needed.

4.3 Real-time architecture — WebSockets, Firestore listeners, modular separation

Sonder is a fundamentally real-time product. Chat messages, presence, typing indicators, shared-itinerary edits, push notifications, and discovery broadcasts all need to arrive immediately. The architecture splits this across two transports chosen for what each is good at.

WebSockets for live conversational traffic. Two endpoints power the live surfaces. `/ws/chat/{session_id}` — one WebSocket per active chat session, carrying messages, typing indicators, seen receipts, and presence updates. `/ws/notifications` — a single global WebSocket per logged-in user, carrying chat notifications, match notifications, comment notifications, and the discovery broadcast events that drive the live travellers strip on the dashboard.

First-message authentication, not query strings. WebSocket endpoints take their auth token in the first JSON message after handshake, never as a query parameter. Tokens in URLs leak through browser history, server logs, and referrer headers. The first-message pattern keeps the credential off the wire.

Heartbeats for presence. Clients send `{"type": "ping"}` every thirty seconds. The backend records `last_seen` to Firestore on each ping. Presence is derived as `(now - last_seen) < 90 seconds` rather than a boolean `online` flag, because boolean flags go stale when WebSocket connections drop unexpectedly. If you have not heard from the user in ninety seconds, you do not know they are online.

Firestore listeners for shared state. The shared-itinerary negotiation page and the presence indicators use Firestore's real-time listener API, which the client subscribes to per-document. Every change to a `shared_itineraries/{id}` document fans out to both participants in milliseconds. Firestore guarantees ordering and delivery semantics that would have required significant engineering to replicate on top of raw WebSockets.

Modular separation across four team-owned folders. The codebase is split into four modules, one per team member, with strict import boundaries. Jahnvi owns schemas, the persona pipeline, and the React frontend. Shreyas owns retrieval, ranking, the matching surface, and the real-time connection manager. Ali owns the LLM clients, the routing engine, all generation prompts, and RAG. Mushahid owns the FastAPI app, the validation engine, the refinement loop, the realtime fan-out layer, and deployment. Cross-module imports are documented per folder. The boundaries are real — a schema change in one folder requires updating the docstrings in folders that import from `shared/schemas.py`. The discipline pays off because each team member can ship independently. Schema changes do not break LLM calls; LLM provider changes do not break the ranker; ranker changes do not break the frontend.

4.4 The shared-itinerary real-time negotiation

The shared itinerary is the surface where two matched co-travellers actually plan the trip together. It is the single most architecturally interesting piece of the system.

Both sides edit one document. When a pair mutually approves, the system clones the chosen itinerary into a new `shared_itineraries/{id}` document with version 0 and both user IDs in `participants`. From that moment, every change goes through the proposal-counter-accept loop rather than direct edit. Direct edits would race; mediated edits are explicit, reviewable, and persisted.

The proposal flow. A user proposes a change — add an activity, replace an existing one, move it to a different day. The proposal writes a `ProposedChange` record with status `proposed` and an `evaluating` activity entry in the itinerary's activity log. The HTTP response returns immediately. The change does not yet exist in the itinerary; it is a pending proposal awaiting evaluation.

The persona evaluator runs asynchronously. Once the proposal is recorded, an `asyncio.create_task` runs the persona's evaluator language-model call. The evaluator is a small-tier prompt that takes the persona, the current itinerary, the recent negotiation history, and the proposed change, and returns a JSON verdict: accept (commit the change) or counter (mint a new `ProposedChange` from the persona with a different activity and a short reason). **There is no hard reject state.** If the persona dislikes the proposal, the system requires it to either accept or offer a counterproposal — never to stonewall. This keeps the negotiation always-collaborative rather than stuck.

Reason-in-message rule. Every counterproposal must include a brief conversational reason: *"hakone feels far after the museum day"*. Without the reason, the counter reads arbitrary. The system prompt enforces this and the validator catches counters that come back reasonless.

Token-Jaccard dedupe on counters. The persona is forbidden from counter-suggesting an activity that already exists in the itinerary, has already been accepted, or has already been rejected. A backend pass normalises titles to lowercase tokens and runs Jaccard similarity (threshold 0.6) against every prior item. If the LLM still returns a near-duplicate counter, the verdict gets flipped to accept with a fallback message (*"yeah okay, let's go with yours"*) rather than circulating the same idea with slightly different wording.

Optimistic locking on every write. Each shared-itinerary document carries a `version` field. Every write checks `client_version == current_version` before committing. A mismatch returns HTTP 409, the client re-fetches the latest state, shows the user a toast (*"Someone else made a change — here's the latest"*), and lets them retry against the updated version. This prevents the classic two-people-editing-simultaneously race condition without silent overwrites.

The activity feed. Every proposal, every counter, every accept, every reject lands in the shared itinerary's activity log with a timestamp and an actor. The frontend renders this as a live feed filtered to entries created after the page opened — so a reload gives a clean visual slate without touching the persistent log. Resolution dedupe strips `evaluating` entries that have a follow-up resolution from the same actor within ninety seconds. Orphan timeout strips `evaluating` entries older than sixty seconds with no follow-up, catching dropped LLM calls.

Finalisation locks the itinerary. Either side can finalise, which sets `finalized_at` and rejects further proposals with HTTP 409. The pair has committed; the trip is the plan.

Bounded by design rather than by a hard turn cap. The negotiation has no maximum number of rounds. The bound comes from the dedupe logic and the no-hard-reject rule: the persona cannot circulate the same idea twice and cannot refuse outright, so the conversation has to progress. Finalisation is the explicit termination.

4.5 The full model inventory

Model	Provider	Role in Sonder
Claude Haiku 4.5	Anthropic	Small-tier conversational surfaces: chat replies, openers, classifiers, *"why this?"* explanations, social posts, proposal evaluation
Claude Sonnet 4.6	Anthropic	Large-tier generative surfaces: itinerary generation, complex refinement, conflict resolution
GPT-4o-mini	OpenAI	Small-tier fallback when Anthropic is unavailable
GPT-4o	OpenAI	Large-tier fallback
text-embedding-3-small	OpenAI	All retrieval embeddings (1536-dim): destinations, activities, traveller profiles, persona text, GoEmotions anchor vectors
gpt-image-1	OpenAI	Synthetic-persona portrait generation, seed-time only
eleven_multilingual_v2	ElevenLabs	Persona voice text-to-speech, with deterministic voice-ID assignment per persona via appearance → accent → gender lookup
GoEmotions cosine classifier	In-process	Emotion scoring over free text via cosine distance against 27 anchor vectors embedded once at process start

Each model does a thing the others cannot do as well or as cheaply. Haiku handles persona-voiced texting at substantially lower cost than Sonnet with no measurable register loss against the validator stack. Sonnet's larger output token ceiling matters for full-trip JSON. The image-generation model's painterly outputs are explicitly biased away from photorealism, which is the right register for the *"Sonder Curated"* disclosure pattern. ElevenLabs offers a multilingual voice library no general-purpose TTS matches. The GoEmotions cosine classifier lives in the same embedding space as everything else, so adding it required no new infrastructure.

4.6 Prompts live in code

Every persona-voiced prompt — chat reply, opener, proposal evaluator, social post, open-trip note, itinerary refinement, validator critics — is a module-level constant in the codebase. Versioning is by git. A prompt change is a reviewable commit. There is deliberately no external prompt store, because the coupling between prompt and surrounding code is explicit. A prompt change that breaks downstream parsing is caught by code review rather than discovered in production.

For larger structural prompt changes, the rollout pattern is to deploy the prompt update paired with a post-hoc normaliser that catches drift from outputs still in flight. The chat opener's *Hey {Name}!* greeting contract is the canonical example. When the format changed mid-deploy, both code paths gained the new prompt rules *and* a wide drifted-greeting matcher that catches outputs from personas that had not yet

observed the new prompt.

4.7 Model updates and the fine-tuning question

Model identifiers are environment-driven. A model bump is a single environment-variable change plus a redeploy. The application code is provider-agnostic and model-id-agnostic. Only the routing layer and the per-provider client know specifics.

Sonder has not used fine-tuning. The team is betting on three less-expensive techniques: prompt engineering with explicit positive and negative examples (every persona-voiced prompt includes both *"Good shapes"* and *"Forbidden"* example blocks), the validator stack as the second line (when prompt engineering misses, the validator catches it, and the validator's failure analysis becomes feedback for the next prompt iteration), and per-user learning at the ranker layer.

Fine-tuning remains a phase-two escape hatch for surfaces where prompt engineering plateaus — most likely the proposal evaluator, where persona-consistent counter-suggestions across long negotiations is the hardest sustained-coherence task in the system.

4.8 Monitoring for drift and regression

Every language-model surface is instrumented. Five top-level metric families drive the analytics dashboard.

User satisfaction. Match found, match approved, match denied, match regenerated, itinerary revision applied, refinement attempt counts. A rising denies-per-match ratio is a leading indicator of a calibration regression. A rising revisions-per-itinerary ratio is a leading indicator of an itinerary-quality regression.

Retrieval quality. Retrieval completion events carry destination count and activity count. A surface returning zero candidates points at a corpus coverage gap before users start abandoning sessions.

Response quality. Validator stack executions per surface carry first-try approval rate, repair count, total latency, and the continuous semantic-genericity score. The genericity score is a leading drift indicator — a creeping rise in mean genericity over weeks is visible long before any individual reply looks bad.

Hallucination rate. Itinerary and persona validation pass rates broken down by issue category. A category climbing in isolation points at a specific regression.

Itinerary completion funnel. Plan started → trip generated → trip saved → trip viewed. The conversion rate between adjacent steps is the product's primary growth metric.

A per-feature distribution observer on the ranker side records mean, variance, and count per (surface, feature) combination. This catches silent scale domination — if one feature is winning most of the combined ranker output because its distribution is wider than others, the asymmetry is visible in the aggregate rather than discovered later through empty engagement metrics.

4.9 The feedback loops running today

Three feedback paths are live in production.

Implicit accept-reject events logged with the candidate's full feature breakdown at the moment of each action. The substrate for phase-two gradient learning.

Explicit free-text revision feedback classified for scope and target (which days, which categories), then routed through either a day-targeted regeneration prompt or a full-itinerary regeneration. The free text is also keyword-mapped to ranker features for that user — *"cheaper"* boosts budget-fit weighting with decay so that repeated similar feedback dampens rather than oscillates.

Live chat signals extracted by the sarcasm-aware scanner after every message, re-ranking the candidate and updating the match score. The persona's reciprocal-approval probability reads the live score at decision time. What a user reveals mid-conversation directly influences whether they end up matched.

Human-in-the-loop oversight is currently lightweight. The team reviews validator-flagged samples through the analytics dashboard weekly. Prompt changes ship through git review. A formal moderation queue is phase-two work — synthetic content does not need it, but user-generated journal entries and feed posts will need it at scale.

---

5. What we learned, what is still broken, and where this goes

5.1 What worked

Information starvation is the most important architectural lever in the system. The discipline of giving each component exactly the slice of context relevant to its narrow task — and merging downstream rather than co-prompting upstream — is what keeps Sonder's outputs surprising rather than averaging toward the mean. The persona inference, the validator, the matcher's policy declarations, and the synthetic-persona blind-writer seeding all use the same pattern.

Async edit-in-place validation outperforms a single large-model call on both latency and cost for conversational surfaces. The reply broadcasts immediately; the validator-driven repair runs asynchronously; the user sees self-correction in place when needed. The pattern generalises to any conversational AI in production.

Equal-weight priors paired with logged-everything infrastructure is the honest move when you do not yet have data. Refusing to hand-tune ranker weights and instead instrumenting every feature observation to disk is what keeps the door open for phase-two learning. Most products tune weights by gut and then cannot reconstruct why. Sonder waits, because the data substrate is being built in the meantime.

5.2 What is still broken or limited

Equal-weight priors leave signal on the table. Honest acknowledgement of a known gap. Resolution is engineering work — accumulate enough feedback events to compute replacement gradients, ship the learning loop.

Synthetic-only safety filters. The same-gender hard filter is enforced against synthetic-persona metadata. Real users joining the matching pool would need a verified gender field. Currently self-reported via the backfill prompt. Scale-up to real-user matching needs identity verification.

No automated test suite for LLM-dependent outputs. Local deterministic tests cover routing, validators, ranking math, and pipeline shapes. The LLM-dependent surfaces are monitored through production telemetry rather than CI-tested, because LLM outputs are hard to assert against deterministically. Regressions can ship and only get caught by aggregate metric drift. Mitigating with validator-stack telemetry is the current answer.

Vector store corpus maintenance is manual. Fifty curated destinations and five hundred curated activities is a finite curation budget. A user picking a city not in the corpus falls back to an *"invent plausible activities for {city}"* prompt that re-introduces the hallucination risk the RAG pipeline normally controls.

Synthetic content does not auto-throttle. The synthetic-agents loop runs at the same cadence regardless of real-user density. Once real-user content reaches a critical mass, the synthetic side should throttle down. Currently a manual configuration change.

5.3 Where this goes next

- **Phase-two ranker learning** replaces equal-weight priors with gradient-learned weights from accumulated accept-reject events.
- **Automated activity-corpus expansion** through scraping plus validator-gated embedding for novel destinations. Removes the prompt-fallback hallucination risk.
- **Multi-modal persona reveal** with persona-voiced audio greetings (the voice infrastructure already exists for chat) or short generated video sequences.
- **Layered prompt-injection defence.** The current input sanitiser is a lightweight regex first line. A language-model classifier on top for the high-stakes free-text fields is the obvious second layer.
- **Cross-candidate diversity in ranking.** The matcher's reranker stage is declared as a no-op in v1, reserved for diversity, fatigue, and sequencing features.
- **Verified identity for real users.** Self-reported gender is a starting point. Scale needs verification.

5.4 References and resources

Models running in production.

- Anthropic Claude — Haiku 4.5 (small tier), Sonnet 4.6 (large tier), validator tier available on either.
- OpenAI — GPT-4o-mini and GPT-4o (fallback provider).

- OpenAI text-embedding-3-small — 1536-dimensional embeddings.
- OpenAI gpt-image-1 — synthetic-persona portrait generation, seed-time only.
- ElevenLabs eleven_multilingual_v2 — persona voice text-to-speech.
- GoEmotions 27-label classifier — in-process cosine over anchor vectors.

Datasets and curated corpora.

- 192 LLM-designed synthetic solo travellers and 18 couples, seeded into the vector store.
- Hand-curated destination corpus (~50 cities) and per-destination activity corpora (~500 activities total).
- Closed twelve-dimension push-pull persona vocabulary.
- Closed eight-key emotional-signature taxonomy with tone-anchored glosses.
- GoEmotions 27-label emotion vocabulary used as anchor vectors.

Academic frameworks the system is built on.

- *Dann, G. (1977). "Anomie, Ego-Enhancement and Tourism." Annals of Tourism Research, 4(4), 184-194.*
- *Crompton, J. L. (1979). "Motivations for Pleasure Vacation." Annals of Tourism Research, 6(4), 408-424.*
- *Demszky, D., et al. (2020). "GoEmotions: A Dataset of Fine-Grained Emotions." arXiv:2005.00547.*

External APIs and services.

- Wikipedia REST API and Wikimedia Commons for destination context and primary imagery.
- Pixabay for travel photography powering the cinematic reveal.
- OpenWeather for current weather by latitude and longitude.
- Nominatim / OpenStreetMap for geocoding.
- ExchangeRate API for currency conversion with a 30-currency hardcoded fallback.

Infrastructure.

- Pinecone (managed vector index, three namespaces).
- Firestore (operational document store, named-database mode).
- Firebase Authentication and Firebase Storage.
- Render (backend hosting).
- Cloudflare Pages with edge functions (frontend hosting and backend proxying).
- VAPID with a service worker (offline web-push notifications).
- Sentry (error telemetry).
- PostHog (product analytics).

Repository. github.com/shreyast36/sonder

Built over a series of sleepless nights by a team that got tired of explaining trips to algorithms.